Ubuntu 常用软件安装与使用

目录 1 终端与路径 2 文件与 Shell 3 APT 与软件源 4 其他安装方式
5 VS Code 与编译工具 6 Git 7 Python 8 SSH 与系统工具 9 常见问题

1. 终端与路径

1.1 打开终端与输入命令

按 **Ctrl + Alt + T** 打开终端。命令由“命令名、选项、参数”组成，输入后按 Enter 执行。

```
student@ubuntu:~/workspace$ ls -la
```

`student` 是用户名，`ubuntu` 是主机名，`~/workspace` 是当前目录；`$` 是提示符。上例实际输入的内容是 `ls -la`。

快捷键	功能
Tab	补全命令、文件名和路径；连续按两次显示候选项
↑ / ↓	调用上一条 / 下一条历史命令
Ctrl + Shift + C / V	在终端中复制 / 粘贴
Ctrl + C	中断前台程序
Ctrl + L	清屏，历史输出仍可向上滚动查看

1.2 系统版本与软件包架构

```
cat /etc/os-release          # 系统版本与代号
dpkg --print-architecture    # 软件包架构
python3 --version            # Python 版本
```

Ubuntu 22.04 的代号是 `jammy`。常见架构：`amd64` 对应 Intel / AMD 64 位电脑，`arm64` 对应 64 位 ARM 设备；下载 .deb 时选择对应架构。

1.3 路径与权限

写法	含义
`/` / `~`	根目录 / 当前用户家目录，如 /home/student
`.` / `..`	当前目录 / 上一级目录
`/etc` / `/usr/bin`	系统配置目录 / 大量系统命令所在的目录
`/home/student/workspace`	以 / 开头的绝对路径；workspace 是相对当前目录的路径

`sudo` 默认以 root 身份执行后面的命令。终端输入密码时不显示字符，输完按 Enter。Linux 文件名区分大小写；路径中含空格时用引号包住。

资料：Ubuntu 命令行教程 | Ubuntu 软件仓库

2. 文件操作与 Shell

2.1 目录切换与文件操作

```
pwd                                  # 当前所在目录
ls -la                               # 列出文件，包含隐藏文件
mkdir -p ~/workspace/robot-demo       # 创建多级目录
cd ~/workspace/robot-demo             # 进入目录
touch main.py                        # 新建空文件；已有文件则更新时间
```

命令示例	作用
`cd ..` / `cd ~` / `cd -`	上一级目录 / 家目录 / 上一次所在目录
`cp main.py main_backup.py`	复制文件；复制整个目录使用 cp -r
`mv main_backup.py old.py`	重命名；目标为目录时表示移动文件
`rm old.py`	直接删除文件，不经过回收站
`cat main.py` / `less main.py`	一次显示全文 / 分页查看；less 中按 q 退出
`grep -n "error" run.log`	在 run.log 中查找 error，并显示行号

以点开头的名称是隐藏文件，例如 `.bashrc`、`.gitignore`。文件管理器中按 **Ctrl + H** 显示隐藏文件；在文件夹空白处右键 → **在终端打开**，终端会直接进入该目录。

2.2 解压文件

```
sudo apt install unzip
unzip project.zip                    # 解压 ZIP
tar -xzf project.tar.gz               # 解压 tar.gz
```

`.zip`、`.tar.gz` 只是压缩格式；解压后可能得到源码、可执行程序或数据。安装说明通常位于 `README.md` 或 `INSTALL`。

2.3 命令连接、输出与注释

写法	含义
`命令1 && 命令2`	命令 1 成功后再执行命令 2
`命令1 | 命令2`	把前一条命令的标准输出交给后一条命令
`命令 > file` / `命令 >> file`	把标准输出覆盖写入文件 / 追加写入文件
`# 说明` / 行末 `\`	# 引出注释；行末反斜杠把下一行接到同一条命令

2.4 PATH、环境变量与 source

```
command -v python3                   # 查看命令实际路径
echo "$PATH"                         # 查看命令搜索目录
export PATH="$HOME/.local/bin:$PATH" # 为当前终端增加搜索目录
source ~/.bashrc                     # 在当前 Shell 重新执行配置文件
```

`PATH` 中的目录按冒号分隔。`export` 的修改作用于当前 Shell 及其子进程；写入 `~/.bashrc` 后，新开的交互式 Bash 终端也会加载。`source 文件` 在当前 Shell 中执行文件，因此能修改当前环境。

资料：Ubuntu 命令行教程 | Bash 手册 | GNU tar 手册

3. APT 软件管理与软件源

APT 从已配置的软件源查询、下载和安装 Debian 软件包，同时处理依赖。依赖是程序运行或编译时需要的其他软件包。

3.1 查找、安装与验证

```
sudo apt update                    # 更新软件包索引
apt search git                     # 搜索软件包
apt show git                       # 查看简介、版本与依赖
sudo apt install git               # 安装 Git
git --version                      # 查看已安装版本
```

安装确认显示 `[Y/n]` 时，输入 y 并回车。多个软件包可以写在同一条命令后面：

```
sudo apt install git curl wget htop tree unzip
```

3.2 更新、查询与卸载

命令	作用
`sudo apt update`	更新可用软件包的索引，不升级软件
`apt list --upgradable`	查看可以升级的软件包
`sudo apt upgrade`	升级已安装的软件包，不等于升级 Ubuntu 发行版
`apt policy git`	查看已安装版本、候选版本及其来源
`dpkg -l git` / `dpkg -L git`	查看安装状态 / 列出该软件包安装的文件
`sudo apt remove tree`	卸载软件，保留包管理器登记的配置文件
`sudo apt purge tree`	卸载软件并清除其包配置；家目录配置通常仍保留
`sudo apt autoremove`	清理自动安装、且已不再被其他已装软件依赖的包

3.3 软件源与镜像站

软件源保存软件包和索引；镜像站提供源内容的同步副本。切换镜像站改变下载地址，不改变 Ubuntu 的发行版本。

图形界面：应用菜单搜索“软件和更新” → **Ubuntu 软件** → **下载自** → **其他站点** → 选择服务器 → 关闭窗口并重新载入软件包信息。

配置文件：Ubuntu 22.04 的主源通常位于 `/etc/apt/sources.list`，额外软件源位于 `/etc/apt/sources.list.d/`。`jammy`、`jammy-updates`、`jammy-security` 分别对应基础仓库、更新仓库和安全更新仓库。

清华镜像配置页可选择 **Ubuntu 22.04 LTS** 后查看对应配置。APT 软件源、pip 软件源和 Snap 商店是三套独立的下载来源。

3.4 启用 universe 软件仓库

```
sudo add-apt-repository universe
sudo apt update
```

`universe` 包含社区维护的软件包。添加仓库后刷新索引，APT 才能检索到其中的软件。

资料：Ubuntu 软件包管理 | Ubuntu 软件更新 | 清华 Ubuntu 镜像配置

4. 其他软件安装方式

4.1 安装方式对照

形式	管理对象	常见用途
APT / .deb	Debian 软件包	系统工具、库文件、桌面软件
Snap	Snap 应用包	桌面应用和独立工具
AppImage	单个可执行应用文件	下载后赋予执行权限并运行
pip	Python 第三方包	算法、数据处理与项目依赖，见第 7 节
源码	源代码及构建文件	通过项目指定的构建工具生成程序

4.2 安装本地 .deb 文件

下载完成后，在文件管理器中打开安装包所在文件夹，右键空白处 → **在终端打开**。以下 `软件包.deb` 替换为实际文件名；输入文件名前几位后按 Tab 可补全。

```
sudo apt install ./软件包.deb
```

`./` 表示当前目录。APT 根据本地 .deb 的依赖声明，从已配置的软件源补齐依赖。`dpkg -i` 也能安装 .deb，但自身不负责联网下载缺失依赖。

卸载时使用**软件包名称**而不是 .deb 文件名，例如 VS Code 的包名是 `code`：

```
sudo apt remove code
```

4.3 Ubuntu Software 与 Snap

应用菜单打开 **Ubuntu Software（Ubuntu 软件）**，搜索软件 → 进入详情 → 安装。命令行以 VLC 为例：

```
snap find vlc                    # 搜索应用
sudo snap install vlc            # 安装
snap list                        # 列出已安装的 Snap
sudo snap refresh vlc            # 更新指定应用
sudo snap remove vlc             # 卸载
```

APT 与 Snap 分别记录安装状态。Snap 安装的程序使用 `snap remove` 卸载；APT 安装的程序使用 `apt remove` 卸载。

4.4 运行 AppImage

在 AppImage 所在目录执行；`Example.AppImage` 替换为实际文件名：

```
chmod u+x Example.AppImage
./Example.AppImage
```

`chmod u+x` 给文件所有者增加执行权限，`./文件名` 运行当前目录中的程序。AppImage 本体无需注册到 APT；移除本体即删除应用文件，用户配置另行保存。

Ubuntu 22.04 上出现 `libfuse.so.2` 缺失时，安装 FUSE 2 兼容库：

```
sudo apt install libfuse2
```

资料：VS Code 的 .deb 安装说明 | Snap 使用指南 | AppImage 运行与 FUSE

5. VS Code 与编译工具

5.1 安装 VS Code

打开 VS Code 下载页，选择 Linux 的 **.deb** 包。`amd64` 电脑选择 x64，`arm64` 设备选择 Arm64。在下载目录中打开终端：

```
ls code_*.deb
sudo apt install ./code_*.deb
code --version
```

* 是通配符。目录中存在多个版本时，在安装命令中写出要安装的完整文件名。安装过程可添加 Microsoft 的 APT 软件源，后续通过 APT 接收更新。

5.2 常用扩展

按 **Ctrl + Shift + X** 打开扩展面板，搜索以下名称并安装。

扩展名称	发布者	作用
Chinese (Simplified) Language Pack	Microsoft	简体中文界面
Python	Microsoft	Python 补全、运行及环境选择
C/C++	Microsoft	C/C++ 补全、代码分析与调试支持
CMake Tools	Microsoft	配置和构建 CMake 项目
Remote - SSH	Microsoft	打开远程设备上的目录并开发

中文界面：在命令面板执行 `Configure Display Language`，选择 `zh-cn` 并重启窗口。

5.3 打开项目与运行文件

```
cd ~/workspace/robot-demo
code .
```

`code .` 把当前文件夹作为工作区打开。在 VS Code 中打开或新建 `main.py`，保存后，在“终端 → 新建终端”中执行：

```
python3 main.py
```

Python 项目使用虚拟环境时，按 **Ctrl + Shift + P** → `Python: Select Interpreter` → 选择项目的 `.venv/bin/python`。创建方法见第 7 节。

5.4 C/C++ 编译工具

```
sudo apt install build-essential cmake gdb pkg-config
g++ --version
cmake --version
```

`build-essential` 提供 GCC / G++、Make 等基础构建工具；`cmake` 生成构建配置；`gdb` 用于调试。编辑器扩展与编译器分别安装。

对根目录包含 `CMakeLists.txt` 且依赖已装齐的项目，在项目目录执行：

```
cmake -S . -B build
cmake --build build -j 4
```

`-S .` 指定源码目录，`-B build` 指定构建目录，`-j 4` 使用 4 个并行编译任务。

资料：VS Code 安装 | C++ on Linux | Python 环境 | CMake 构建

6. Git：下载代码与版本管理

Git 在本地记录代码版本；GitHub、GitLab 等平台保存远程仓库。`git clone` 下载的是仓库内容，后续依赖安装和编译由项目的 README 说明。

6.1 设置提交身份

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

姓名和邮箱写入每次提交的作者信息，并不是远程平台的登录凭据。`--global` 表示对当前用户的所有仓库生效。

6.2 下载仓库

复制项目页面上的 HTTPS 仓库地址。下面的 `OWNER`、`REPO` 分别替换为实际的所有者和仓库名。

```
mkdir -p ~/workspace
cd ~/workspace
git clone https://github.com/OWNER/REPO.git
cd REPO
```

项目包含子模块时，在仓库目录补齐子模块内容：

```
git submodule update --init --recursive
```

6.3 保存本地修改

工作区 → 暂存区 → 本地仓库：编辑文件后，用 `git add` 选择进入下一次提交的内容，再用 `git commit` 保存为一个版本。

```
git status                           # 查看文件状态
git diff                             # 查看工作区尚未暂存的修改
git add main.py                      # 暂存指定文件
git commit -m "Update main program"  # 保存本地提交
git log --oneline -5                 # 查看最近 5 次提交
```

上述 `main.py` 指仓库中实际修改的文件。已有的普通项目文件夹先执行 `git init`，即可建立本地仓库。`commit` 只保存在本机；上传到远端使用 `push`。

6.4 远程同步与忽略文件

命令	作用
`git remote -v`	查看远程仓库地址
`git pull --ff-only`	获取并快进更新当前分支；分支已分叉时停止
`git push`	将本地提交上传到已配置的上游分支

在项目根目录创建 `.gitignore`，写入不需要进入版本库的路径。例如：

```
.venv/
__pycache__/
build/
*.pyc
```

`.gitignore` 只控制尚未被 Git 跟踪的文件，不会自动移除已经提交的文件。

资料：Git 初次配置 | Git 仓库与提交 | Git 子模块

7. Python 与虚拟环境

Python 执行程序；**pip** 安装第三方包；**venv** 为项目建立独立的包安装目录。`python -m pip` 使用当前 python 对应的 pip。

7.1 安装工具、创建并激活环境

```
sudo apt install python3-pip python3-venv
cd ~/workspace/robot-demo
python3 -m venv .venv
source .venv/bin/activate
```

激活后提示符前通常出现 (.venv)。.venv 是环境目录；已有环境再次使用时，只需执行激活命令。每个新终端分别激活。

```
python -c "import sys; print(sys.executable)"
```

解释器路径应位于项目的 .venv 内。虚拟环境沿用创建它的 Python 版本。

7.2 安装与管理第三方包

```
python -m pip install numpy
python -c "import numpy; print(numpy.__version__)"
```

命令	作用
`python -m pip list`	列出当前环境中的已安装包
`python -m pip show numpy`	查看版本、安装位置和依赖
`python -m pip install --upgrade numpy`	升级指定包
`python -m pip uninstall numpy`	卸载指定包
`python -m pip check`	检查已装包的依赖是否满足

指定版本的写法为 包名==版本号；例如项目要求 NumPy 1.26.4 时，使用 `python -m pip install "numpy==1.26.4"`。

7.3 安装项目依赖与记录环境

```
python -m pip install -r requirements.txt
python -m pip freeze > requirements.lock.txt
```

第一条读取项目依赖；第二条导出已装包及版本，包含间接依赖。恢复记录的环境使用 `python -m pip install -r requirements.lock.txt`。

7.4 pip 镜像、运行程序与退出

单次安装改用清华 PyPI 镜像：

```
python -m pip install numpy \
  -i https://pypi.tuna.tsinghua.edu.cn/simple
```

-i 只改变本次安装的包索引地址，不修改 APT 软件源。运行程序及退出环境：

```
python main.py
deactivate
```

deactivate 退出环境；.venv 中的文件继续保留。

资料：Python 虚拟环境与 pip | pip 命令参考 | 清华 PyPI 镜像

8. SSH、文件传输与系统工具

SSH 用于从本机登录另一台电脑、服务器或机器人。发起连接的一端安装客户端，被连接的一端运行 SSH 服务。

8.1 在远程设备上启用 SSH 服务

```
sudo apt install openssh-server
sudo systemctl enable --now ssh
systemctl is-active ssh
hostname -I
```

`active` 表示服务已运行；`hostname -I` 列出设备的 IP 地址。以下例子使用远程用户 `robot` 和地址 `192.168.1.100`，操作时替换为实际值。

8.2 在本机发起连接

```
sudo apt install openssh-client
ssh robot@192.168.1.100
```

首次连接会显示主机指纹，确认后输入 `yes`，该主机记录保存到 `~/.ssh/known_hosts`。随后输入**远程账号**的密码。登录后，终端中的命令在远程设备上执行。

```
exit                                # 退出远程登录
```

指定其他端口使用 `ssh -p 2222 robot@192.168.1.100`，其中 2222 为示例端口。

8.3 使用 scp 传输文件

以下命令均在**本机终端**执行：

```
scp main.py robot@192.168.1.100:~/
scp robot@192.168.1.100:~/run.log ./
scp -r ./robot-demo robot@192.168.1.100:~/
```

依次为：上传文件、下载文件、上传目录。远程路径写作 `用户名@IP:路径`；冒号后 `~/` 是远程用户的家目录。`scp` 指定端口使用大写 `-P`。

8.4 常用系统与设备检查

```
sudo apt install htop tree curl wget usbutils
```

命令	读取的信息
`htop`	CPU、内存与进程；按 q 退出
`df -h` / `du -sh 路径`	文件系统剩余空间 / 指定文件或目录占用
`free -h`	内存与交换空间；available 为可供应用使用的估计值
`ip -br addr`	网卡名称、状态与 IP 地址
`lsusb`	系统枚举到的 USB 设备
`tree -L 2`	当前目录下两层文件结构
`curl -I https://ubuntu.com`	请求响应头，查看 HTTP 状态与重定向等信息

下载文件可用 `wget 下载地址`；指定保存文件名用 `wget -O 文件名 下载地址`。

资料：Ubuntu OpenSSH | OpenSSH scp | Ubuntu 命令手册

9. 常见问题与处理命令

9.1 软件安装与命令执行

报错或现象	检查与处理
`command not found`	用 `command -v 命令名` 检查搜索结果；用 `echo "$PATH"` 查看搜索目录。已安装但不在 PATH 中的程序可用完整路径启动。
`Unable to locate package`	执行 `sudo apt update` 更新索引，再用 `apt search 关键词` 查询实际包名；`apt policy 包名` 查看候选版本。第三方包需要相应的软件源。
`dpkg was interrupted`	执行 `sudo dpkg --configure -a`，继续完成尚未配置的软件包。
依赖未满足 / broken packages	执行 `sudo apt --fix-broken install`，让 APT 计算依赖修复方案。
`Could not get lock`	APT / dpkg 被另一个进程占用。报错包含 PID 时，可用 `ps -p PID -o pid,cmd` 查看任务；待该软件管理任务结束后重新执行。
`Permission denied`	对文件执行 `ls -l 文件名` 查看拥有者、所属组和权限。脚本缺少执行权限时用 `chmod u+x 文件名`；`r/w/x` 分别表示读、写、执行。
`No space left on device`	用 `df -h` 查看剩余空间，`du -sh 路径` 定位占用，`df -i` 检查 inode 是否用尽。

9.2 Python、SSH 与设备权限

报错或现象	检查与处理
`ModuleNotFoundError`	用 `python -c "import sys; print(sys.executable)"` 确认解释器，再用 `python -m pip show 包名` 检查包是否安装到这个环境。VS Code 中同步选择同一解释器。
`externally-managed-environment`	当前解释器由系统包管理器管理。按第 7 节创建并激活 `.venv`，再执行 `python -m pip install 包名`。
SSH：`Connection refused`	远端对应端口未接受连接。在远端检查 `systemctl is-active ssh` 和 `ss -ltn` 的监听端口，并核对连接端口。
SSH：`Connection timed out`	检查目标 IP、网络路由和防火墙；本机用 `ip route` 查看路由，用 `ssh -v 用户名@IP` 查看连接停在哪一步。
串口设备无访问权限	用 `ls -l /dev/ttyUSB0` 查看设备所属组。若组为 `dialout`，执行 `sudo usermod -aG dialout "$USER"`，注销并重新登录后生效。

软件包名、命令名和 Python 导入名不一定相同。例如 `openssh-client` 提供 `ssh` 命令；`opencv-python` 对应 `import cv2`。安装与排错时分别核对。

资料：APT / dpkg 手册 | Python 系统管理环境 | OpenSSH 文档